Curtin College | in association with Curtin University

Curtin College
DSA1002 Data Structure and Algorithm
Trimester 3, 2023

Assignment

Weight: 40% of the unit

Assignment Location: Assignment is uploaded under **Assessments section (Final Assessment – Assignment)** on unit Moodle page**.**

Answer Format. When you write an answer, clearly indicate the relevant question number/letter. Include your name and student ID at the start. Also add appropriate comments to code files to indicate author name and student ID. Detailed submission guidelines can be found below in section 3.

Timeframe. You have 14 days (336 hours) to complete and submit your answers, from **09:00 am on 5th Jan 2024** until **09:00 am on 19th Jan 2024** (UTC+8). You may schedule your work within this period. **However, late submissions are not allowed** (also check late submission policy in unit outlines).

Submission. Submit your answer document(s) to the "**Final Assessment– Assignment**" area on Moodle under assessments section. You must verify that your submission was successful. Correctly submitting is entirely your responsibility.

Report: There is separate submission point for Report, make sure to submit report part on this submission point.

Implemented Code: There is separate submission point for Implemented Code, make sure to submit code part on this submission point.

Reference Material. This is an OPEN BOOK and OPEN COMPUTER assignment. You may refer to any written material, including your notes, course materials, books, websites, Unit Moodle page recordings etc. However:

- You must complete this assignment entirely on your own.
- You should answer all questions in your own words and code.
- You can use pseudo code and algorithms provide in the unit slides (Moodle page) for your implementation.
- During the assignment, you may not communicate with any other students/anyone helping.
- Your answer document will be checked by text matching software for signs of cheating,collusion and/or plagiarism.
- The assignment questions have been designed such that any two students, working independently,should not produce the same answers.
- The coding part of this assignment can be submitted in either python/java.

- Find detailed granular level marking rubrics at the end of the assignment.

****** READ COMPLETE DOCUMENT BEFORE STARTING******

1. Overall Assignment Description

In practicals you have implemented and learned about a number of algorithms and ADTs and will be implementing more of these in the remaining practicals. In this assignment, you will be making use of this knowledge to implement a system to explore and compare a variety of ADT implementations. Feel free to re-use the generic ADTs from your practicals. However, remember to self-cite; if you submit work that you have already submitted for a previous assessment (in this unit or any other) you have to specifically state this. **Do not use the Java/Python implementations of ADTs – if in doubt, ask.**

Introduction

In this assignment, you'll utilize your knowledge of algorithms and abstract data types (ADTs) to develop a system aimed at optimizing traffic flow in urban areas. The system will employ various algorithms and data structures to alleviate congestion and enhance transportation efficiency.

Problem Description

Traffic congestion in urban environments poses significant challenges. This assignment aims to create a system that leverages technology to streamline traffic flow, mitigate congestion, and improve overall transportation efficiency.

Task 1: Constructing a Traffic Network

Create a graph representing the road network in an urban area. Nodes in the graph will denote different intersections or road segments, while edges will represent connections or roads between these locations. Utilize adjacency lists to model the graph structure.

Input: Utilize a file (**traffic_network.txt**) containing data on road connections, traffic flow, and distances between different locations in the map.

Output: Display the adjacency list representing the road network.

Task 2: Traffic Flow Optimization

Implement algorithms such as BFS and DFS Algorithm to optimize traffic flow and identify efficient routes between different intersections. Use collected data on traffic volume, road capacity, and real-time traffic conditions for route planning. BFS will be used to find shortest path between locations and DFS can be used to explore all locations and path in the graph.

Input: Road network graph from Task 1 and traffic data (**traffic_data.txt**) containing information on traffic volume, road capacity, and real-time congestion, for each path in the graph. Also take two locations as input.

Output: Optimized traffic route between two locations entered. This should also consider related information for each edge in the path.

Task 3: Dynamic Traffic Management

Implement functionalities to dynamically (means you can input new locations and paths) update the traffic network, add new roads or intersections, modify traffic flow restrictions,

and search for specific routes within the urban area. Develop an interactive menu for these operations.

Task 4: Efficient Data Handling

Utilize hash tables to efficiently manage traffic-related data, such as traffic volumes, congestion levels, and road conditions at various intersections or road segments.

You can add the edges in a hash table (from traffic_data.txt), this will help you to retrieve related data for each path quickly while accessing for shortest path calculation. (Hint: you can use start point of edge given in the text file as key to insert in the hash table)

Compare the efficiency of hash tables against array and linked_list (double ended single linked list) for managing traffic data.

Task 5: Prioritized Traffic Control

Implement a priority queue (heap) to prioritize traffic management strategies based on congestion levels, emergency situations, or roadwork. This will assist in deploying traffic control measures effectively. You can insert the edges from textfile (**traffic_data.txt)** to a heap and it will help you to find out highest congestion edge, also sorting edged based on their congestion level. (Hint: use traffic volume as a priority to insert data in heap(max))

Task 6: Advanced Traffic Optimization

Research and implement advanced algorithms (e.g. Dijkstra's algorithm) beyond the simple DFS and BFS. You can change your path finding algorithm used in Task 2 with this algorithm.

Task 7: Program Implementation and Testing

Develop a program to manage traffic flow in urban areas, optimize routes, alleviate congestion, and improve transportation efficiency. Include an interactive menu for user interaction, error handling.

2 Project Report

A project report of minimum 8-10 pages should be submitted (pdf format) including following details:

Usage information:

- Introduction: describing basic introduction of your program (software).
- Dependencies: any libraries required to use the program (software).
- Terminologies and abbreviations used in the code.
- Future directions: suggested future improvements.

Class UML Diagrams:

- Readme file, describing related information[1].
- Complete UML class diagrams of the classes used for implementation.
- A complete association of classes/objects (i.e., class relationship).
- Complexity analysis of all operations performed by the software (eg. Load data, etc.).
- Traceability matrix of feature implementation and testing of your code[2].

Comments on Code: it is suggested to add detailed comments in your code.

References: (if any) all materials should be referenced Chicago referencing style.
[1]Example good Readme files can be found here. (https://github.com/matiassingers/awesome-readme)
[2]Traceabiliy matrix help. (https://www.youtube.com/watch?v=8_5xZAXdS_A)

3 Submission

Submit electronically through Moodle unit page under assessments section ("**Assessment 2 – Assignment 1**").
You should submit a single file, which should be zipped (.zip) or tarred (.tar.gz). Check that you can decompress it on the lab/personal computers. Your work will be tested on lab/computer other than your PC so try to check your code on other PCs too. The file must be named DSA_Assignment_1_<student id>, use underscores instead of the spaces in the file name.
The file should contain following deliverables:

- Your code. This means all python/java files needed to run your program. Do include code provided to you as part of the assignment if that is required to run your program. Do not include .class files or anything else that is not required to recompile python/java files.
- README file including short descriptions of all files and dependencies, and information on how to run the program (see section 2).
- Your program (software) test harnesses. One of the easiest ways for us to be sure that your code works is to make sure that you've tested it properly.
- Documentation and Report for your code (Project Report)

Please verify that your submission is correct and not corrupted. You may make multiple submissions, only your last one will be marked. **However, late submissions are strictly not allowed** (also check late submission policy in unit outlines).

4 Marking Criteria

The assignment will be marked based on the following breakdown of the submission:

Code Implementation: (20 Marks) Code should be demonstrated during the tutorial to achieve this requirement. The code should be appropriately written, as ADTs with comments. The code developed will be tested against different tests (as per requirements given in Questions 1).

Project Report: (10 Marks) A minimum 8-10-page report based on information describe in section 2.

Code Testing and Demonstration: (10 Marks) Code should be implementable, and testable with the test harness. Also should be demonstrated as per given schedule.

5 Academic Integrity

Please see the Coding and Academic Integrity Guidelines on unit Moodle page.

In summary, this is an assessable task. If you use someone else's work or assistance to help complete part of the assignment, where it's intended that you complete it yourself, you will have compromised the assessment. You will not receive marks for any parts of your submission that are not your own original work. Further, if you do not reference any external sources that you use, you are committing plagiarism and/or collusion, and penalties for academic misconduct may apply.

Curtin college also provides general advice on academic integrity at https://www.curtincollege.edu.au/content/dam/navitas/upa/curtin/pdfs/academic-integrity-policy.pdf

The unit coordinator may require you to provide an oral justification of, or to answer questions about, any piece of written work submitted in this unit. Your response(s) may be referred to as evidence in an academic misconduct inquiry.

Granular Marking Rubrics

Code Implementation (20 Marks)

- Correctness of Code: (6 Marks)
    a) Demonstrates correct implementation of the graph representing the road network using adjacency lists.
    b) Accurate implementation of BFS, DFS, and other algorithms for traffic flow optimization.
    c) Proper integration of traffic data (from traffic_data.txt) to optimize traffic routes between different locations.
- Efficient Data Handling: (4 Marks)
    a) Implementation and utilization of hash tables for efficient management of traffic-related data.
    b) Comparison of efficiency between hash tables, arrays, and linked lists for managing traffic data, supported with analysis.
- Prioritized Traffic Control: (4 Marks)
    a) Implementation of a priority queue (heap) for prioritizing traffic management strategies based on congestion levels, emergency situations, or roadwork.
    b) Proper insertion of traffic data into the heap based on congestion levels or traffic volume.
- Advanced Traffic Optimization: (6 Marks)
    a) Successful implementation and integration of advanced algorithms (e.g., Dijkstra's algorithm) for traffic flow optimization.
    b) Comparative analysis and demonstration of improved performance using advanced algorithms over simple DFS/BFS.

Project Report (10 Marks)

- Usage Information: (2 Marks)
    a) Clear introduction explaining the program's purpose and functionalities.
    b) Dependencies and required libraries detailed for using the software.
- Terminologies and Future Directions: (2 Marks)
    a) Clear explanation of terminologies and abbreviations used in the code.
    b) Well-articulated suggestions for future improvements in the software.

- Class UML Diagrams and Complexity Analysis: (6 Marks)
    a) Complete UML class diagrams depicting classes used for implementation and their associations.
    b) Thorough complexity analysis of all operations performed by the software, e.g., load data, pathfinding, etc.

- Traceability Matrix and Comments on Code: (6 Marks)
    a) A detailed traceability matrix showcasing feature implementation and testing of the code.
    b) Comprehensive comments within the code, aiding understanding and readability.

Code Testing (10 Marks)

- Testability and Correctness: (5 Marks)
    a) Demonstrated implementability and testability of the code with a comprehensive test harness.
    b) Successful execution and correctness of the code against various test scenarios.
- Error Handling and Robustness: (5 Marks)
    a) Proper error handling mechanisms incorporated within the code.
    b) Robustness demonstrated against unexpected inputs or scenarios.